SC2006 Lab 3 — 3.3 AI-based project initialisation

Project: PolyNTU

This report covers both exercises in section 3.3 of the Lab 3 manual.

Part	Manual sections	Contents
Exercise A	3.3.1, 3.3.2	Technology stack re-run with the updated class diagram, and comparison with Lab 2
Exercise B	3.3.3, 3.3.4	AI-generated project skeleton and critical evaluation

The AI-generated skeletons are submitted separately from the application source:

- `SC2006_Lab3_3.3.3_AI_Skeleton.zip` — the 3.3.3 deliverable, generated from the class diagram as a PNG
- `SC2006_Lab3_3.3.4_AI_Skeleton_Followup_PUML.zip` — the controlled follow-up described under 3.3.4, generated from the PlantUML source with an identical prompt

Lab 3 — Exercise A: Technology stack re-run (3.3.1, 3.3.2)

AI used: Claude Opus 5.5
Chat: New incognito chat, so account memory and past chats were not available
Date: 25 October 2026

Inputs compared with Lab 2

	Lab 2 (3.4.2)	Lab 3 (3.3.1)
Requirements	`requirements.md`	`requirements.md` (same file)
Diagram	Use case diagram	Updated design class diagram (`lab3/class-diagram.puml`, 39 classes)
Prompt	Prompt 1 below	Prompt 1 below, word for word

The prompt and the requirements document are unchanged. The only input that changed is the diagram.

1. Prompt 1

Attached: `requirements.md`, `class-diagram.puml`

> Given the nature of the platform needing quick update and fetch times for pricing, as well as considerations of scalability, what tech stacks would you recommend, and what technologies/libraries should we consider to optimise/sustain the platform

2. Response 1

> Your scale is small: 500 users, 100 concurrent, 50 markets. The hard problems here are correctness under concurrency, money-safe arithmetic, and honest real-time updates. Throughput isn't one of them. A single well-built service on Postgres covers NFR-1 and NFR-3 with lots of headroom. Most of the risk is in over-engineering, not under-provisioning.
>
> Your class diagram (AppState::new(), ApiRouter::router(), Store sharing a pool, EventRuntime::runListener()) looks like Rust with Axum, and Postgres LISTEN/NOTIFY behind the event bus. If that's what you've built, keep it. It's a strong fit for this problem, and a rewrite now would cost more than any stack choice could save. If I've misread it, the same architecture works in TypeScript (Fastify + Postgres) or Go.
>
> **Recommended stack**

Layer	Recommendation	Why
API server	Rust: axum, tokio, tower-http, sqlx	Async, low latency, compile-time-checked SQL
Database	PostgreSQL 16+, single primary	Real transactions, row locks, constraints, LISTEN/NOTIFY
Money math	rust_decimal or integer minor units	Never store balances as floats
Realtime	Server-Sent Events (axum has Sse built in)	One-way price push; trades already go over POST
Event fan-out	Postgres LISTEN/NOTIFY → in-process tokio::sync::broadcast	Works across multiple app instances without Redis
In-process cache	moka	Market list and series metadata; no Redis needed yet
Auth	argon2 (argon2id), HttpOnly session cookies, tower_governor for rate limits	Meets NFR-4; prevents login and confirm spam
Background jobs	Your BackgroundWorker + a Postgres advisory lock	Only one instance runs the scheduler
Photo storage	S3-compatible (Cloudflare R2, or MinIO for local dev), presigned uploads	Keeps large uploads off your API server
Frontend	React + Vite (or SvelteKit), TanStack Query, uPlot or TradingView lightweight-charts	SSE events update the query cache directly; charts stay light
Mobile	PWA (installable, Geolocation API)	Confirmers need quick launch at a bus stop; no app store
Observability	tracing + OpenTelemetry, Prometheus metrics, Sentry	You can't prove NFR-1 without p95 trade latency
Testing	proptest, testcontainers (real Postgres), k6 for load	Market maker invariants, real locking behaviour, NFR-1 and NFR-3 evidence
Hosting	One container plus managed Postgres in ap-southeast-1 (Singapore)	Hosting in the US would use up a big share of your 1 s budget in round trips

Explicitly avoid: Kafka or any message broker, microservices, Kubernetes, MongoDB (you need multi-row ACID transactions), and anything blockchain. Each adds operational burden and gives nothing at this scale.

What actually matters

1. Serialize trades per market, not globally. An LMSR trade is a read-modify-write on the market's outstanding shares. Do the whole thing in one transaction: SELECT ... FROM market_instance WHERE id = $1 FOR UPDATE, compute the cost, debit the wallet, update positions and outstanding shares, insert the trade, then NOTIFY. Trades on different markets don't block each other. A single hot market can still do hundreds of trades per second, far more than you'll ever see.

You already have instanceVersion on Trade and a quote/execute split in TradingControl. Use them for optimistic validation: the quote carries a version, and execute rejects or requotes if

the version moved past a slippage tolerance. Without this, users get filled at a price they never saw.

2. Make every trade idempotent. recoverReceipt() and resumePendingTrade() suggest you're thinking about this already. Have the client generate an idempotency key per order, and put a unique constraint on it. On flaky campus Wi-Fi, users retry, and a retry must never become two fills.

3. Do the LMSR arithmetic carefully. The cost function is $C(q) = b \cdot \ln \Sigma \exp(q_i/b)$. It's fine to compute this in f64, but use the log-sum-exp trick (subtract max q_i/b first) or it overflows once markets get lopsided. Convert to fixed-point at the boundary, and round so the user pays the higher value and receives the lower one. That stops anyone farming rounding errors with buy/sell loops.

Write property tests that encode the invariants:

- Prices sum to 1.
- Buying then immediately selling never yields a profit.
- The market maker's worst-case loss never exceeds $b \cdot \ln(n)$.
- Total points across all accounts is conserved; only explicit grants change it.

These tests will catch more real bugs than any amount of load testing.

4. Treat the ledger as double-entry and enforce it in the database. LedgerTransfer with from/to accounts is the right model. Add CHECK (balance >= 0) and make reconcile() assert that the sum of balances equals the sum of grants. The database is your last line of defence when application logic has a bug.

5. Make the audit log immutable in the database, not just by convention. NFR-8 says immutable, so REVOKE UPDATE, DELETE on the trade, evidence, and admin-log tables from the app's DB role. Add an append-only trigger as a second layer. If you want something you can show assessors, add a hash chain (each row stores the hash of the previous row). It's cheap and makes tampering detectable.

6. Real-time updates: use SSE and coalesce them.

- Why SSE over WebSockets: prices only flow server to client. SSE auto-reconnects, supports Last-Event-ID, and passes through proxies more easily.
- Coalesce per market: emit at most one update every 150–250 ms per market. Otherwise a burst of trades means N trades × M viewers worth of messages.
- Treat NOTIFY as a doorbell: it isn't durable and payloads cap at 8 KB. On reconnect, the client refetches a snapshot.
- The trader's own screen doesn't need SSE: the POST response should already contain the new prices, which satisfies NFR-1 directly.
- Proxy settings: if you put nginx in front, turn off proxy buffering for the SSE route, or updates arrive in delayed batches.

7. Photos: strip EXIF metadata on upload. This is the one I'd flag as a real bug-in-waiting. Phone photos embed GPS coordinates in their EXIF data. Serving queue photos as-is would break NFR-9. Re-encode server-side (the image crate), strip metadata, cap the dimensions, and keep the original timestamp in your own database column instead.

8. Be honest about the limits of GPS. Browser geolocation is self-reported, so accuracy ≤ 50m (NFR-5) is a client claim and anyone with dev tools can spoof it. Your real anti-fraud defences are server-side:

- The multi-confirmation rule.
- Per-account and per-IP rate limits.
- Reputation weighting.
- Flagging accounts whose confirmations cluster suspiciously.

Write that limitation into your report. Assessors tend to respond well to a team that acknowledges it.

9. Make scheduled jobs idempotent. spawnDueBrackets() should rely on a unique constraint like (series_id, bracket_start), so a double-fire after a restart is harmless. Wrap the tick in pg_try_advisory_lock so you can scale to two instances later without duplicate markets. For the external bus feed behind NtuBusAdapter, add timeouts, caching, and a fallback so a dead upstream API voids markets gracefully instead of hanging them.

Before you go further

Your requirements.md and class diagram have drifted apart. The requirements describe geofenced crowd confirmations (FR-4.3–4.5), while the class model resolves through AdminAuthority, CreatorSignedAuthority, and ExternalResolverAuthority, with no Confirmation or QueueReport entity. Either is defensible. But a marker tracing FR-4.5 → class → test case will hit a dead end, so reconcile them, or log the design change with new requirement IDs as your own doc recommends.

For Lab 4, a k6 script with 100 virtual users holding SSE connections while trading, asserting p95 trade round-trip under 1 s, gives you direct, traceable evidence for NFR-1 and NFR-3 (e.g. TC-NFR1-01).

---

3. Prompt 2 (follow-up, same chat)

Attached: `SC2006_Lab_2.pdf` (the Lab 2 prompt, the Lab 2 response, and the team's Lab 2 opinion on Redis)

> This is another set of recommendations from an earlier version. We eventually settled on building on a Rust stack. Do you agree with the changes?

4. Response 2

> Mostly yes. The move to Rust is defensible and the carry-over from the earlier design is sound. Make sure your justification for Rust is the right one, though, and correct one point in your Redis reasoning before it goes into a report.
>
> **Why Rust is a good choice here (and why it isn't)**

Be careful not to write "we chose Rust for performance." At 500 users and 50 markets, the Node/NestJS stack in that earlier response would also have met NFR-1 comfortably. An assessor who knows this will see through a performance-only argument.

The stronger justification is correctness:

- Money can't be a float by accident. Rust makes you choose a type like rust_decimal.
- The compiler enforces your state machine. Market states (Open/Closed/Resolving/Resolved/Voided) and resolution authorities become enums, and every match must handle every case, so a forgotten transition fails to compile instead of corrupting a ledger.
- Your SQL is checked against the real schema. sqlx macros verify queries at build time.
- You deploy one small binary. That suits free-tier hosting with tight memory limits.

The honest costs are a steeper learning curve, slower compile-edit loops, and async Rust's occasional lifetime fights. On a course timeline that's a real risk. A few things keep it manageable:

- Keep it a monolith.
- Use bacon or cargo-watch for fast feedback while coding.
- Enable sqlx's offline mode so CI doesn't need a live database.
- Keep the frontend in TypeScript, where the rest of the team is likely more productive.

What changes from the earlier recommendation

Earlier (Node)	With Rust	Notes
Postgres row lock per trade	Same	The best advice in that response. It carries over unchanged.
Prisma	sqlx	Compile-time-checked SQL. Migrations via sqlx migrate.
Socket.IO rooms	SSE via axum's Sse	socketioxide exists, but you don't need bidirectional sockets. Trades go over POST.
Redis for cross-instance broadcast	Postgres LISTEN/NOTIFY	Every instance already connects to the same DB, so NOTIFY reaches them all.
BullMQ delayed jobs (45 s window, 10 min dispute)	Deadlines stored as timestamps, checked by BackgroundWorker::tick()	More robust: a restart can't lose a queued job, because the deadline lives in the DB. Tick granularity (e.g. 2–5 s) bounds how late a resolution fires.
bcrypt	argon2 (argon2id)	Current OWASP preference. bcrypt is still acceptable.
JWT sessions	Opaque session token, hash stored server-side	Your Account.tokenHash already implies this. Unlike JWTs, these can be revoked instantly.
Turf.js geofencing	geo crate, or plain haversine in SQL	Check first whether you still need it. Your class diagram moved to admin, creator-signed, and external resolution, and no Confirmation entity remains.

Earlier (Node)	With Rust	Notes
Vercel + Railway/Fly in Singapore, k6, lightweight-charts, no blockchain	Same	All still correct.

On your Redis opinion

Your conclusion is right, but the reasoning needs a correction. The earlier response wasn't proposing Redis as a cache. It proposed Redis as the authoritative live state, updated atomically by a Lua script. In that design nothing is stale, because there is no older copy for Redis to lag behind. Staleness is a concern for read-through caches with a TTL, which is a different pattern.

The real problem with that design is the write-behind queue. Redis confirms the trade to the user, then Postgres is written afterwards. If that write fails or the queue is lost in a crash, prices and ledger disagree. You then have two sources of truth and need reconciliation logic to repair them. That's acceptable for a trading firm with a dedicated ops team. For a student project, it's a source of bugs you can't afford.

So for your report, I'd frame it as: "Redis-as-state removes lock contention but introduces a durability gap between price state and the ledger. At our scale, per-market row locks in Postgres never contend meaningfully, so we keep a single source of truth." Then back it with a k6 result showing p95 trade latency well under 1 s. That turns "we think it's overengineering" into a measured decision, which is the version that earns marks.

---

5. Answers to 3.3.2

How similar are the recommendations?

They are different in their technology choices and mostly the same in their architecture. The Lab 2 response recommended Node.js with TypeScript and NestJS, Prisma, Socket.IO and Redis with BullMQ. The Lab 3 response recommended Rust with Axum, sqlx and Server-Sent Events, and advised against adding Redis at all. Underneath the different tools, both responses gave the same core advice: PostgreSQL as the single source of truth, per-market row locking with `SELECT ... FOR UPDATE`, a single monolithic service, k6 load testing against NFR-1 and NFR-3, and no blockchain or microservices.

(1) What in the inputs explains the difference?

Our prompt and our requirements document were identical in both runs, and neither states a technology preference. The only change was the diagram. In Lab 2 we supplied a use case diagram, which shows actors and goals and says nothing about implementation. In Lab 3 we supplied our design class diagram, whose class and method names carry implementation detail: `AppState.new()`, `Store.connect()`, `ApiRouter.router()`, `BackgroundWorker.tick()` and `EventRuntime.runListener()`. The response named these classes and said they looked like Rust with Axum before recommending it, so we attribute the change of language to the diagram. The same detail explains why it dropped Redis and BullMQ: our diagram already shows an event bus,

an event listener and a scheduled background worker, and the response built on those rather than adding new infrastructure. The class diagram also let it anchor advice to specific elements of our design, such as `instanceVersion` for quote validation, `recoverReceipt()` for idempotent trades and `reconcile()` for ledger checks, which the Lab 2 response could not do. Some smaller differences, such as which charting library it named, are more likely to be normal variation between responses, since we ran each prompt only once.

(2) Does the AI agree with our change of stack?

We moved from the Lab 2 recommendation to a Rust stack, so we asked the AI to compare the two. It mostly agreed. It said the switch is defensible, but warned against justifying it on performance, since the Node stack would also have met NFR-1 at our scale. It argued that the stronger justification is correctness: typed money values, market states and resolution authorities enforced as enums by the compiler, and SQL checked against the schema at build time. It also named the costs, which are a steeper learning curve and slower compile times on a course timeline. It also challenged the reasoning in our Lab 2 opinion on Redis. We had objected to Redis because a cache can serve stale prices. The Lab 2 response did label its Redis row "Cache", so we read it that way. However, its scale-out pattern used Redis as the authoritative live state rather than as a cache, so staleness was not the real issue. The stronger objection is the write-behind gap: Redis confirms a trade to the user before PostgreSQL records it, so a crash in between leaves the price state and the ledger disagreeing. Our conclusion to avoid Redis stands, and this is now our reason for it.

Lab 3 — Exercise B: AI-generated project skeleton (3.3.3, 3.3.4)

AI used: Claude Opus 5.5, same model for both runs
Date: 25 October 2026
Artifacts: `SC2006_Lab3_3.3.3_AI_Skeleton.zip` (run 1), `SC2006_Lab3_3.3.4_AI_Skeleton_Followup_PUML.zip` (run 2)
We ran the exercise twice. Run 1 is the exercise as specified, with the class diagram supplied as a PNG. Run 2 is a controlled follow-up with the identical prompt in a new chat, supplying the PlantUML source of the same diagram instead, to establish whether the omissions in run 1 came from the input format. No other variable was changed.

---

1. Prompt

The same prompt was used for both runs. The only difference between runs was the file attached: `class-diagram.png` for run 1, `class-diagram.puml` for run 2.

> I'm attaching the design class diagram for PolyNTU, a campus prediction-market web application.
>
> Generate a minimal project skeleton for it. Our stack is Rust for the backend and React with JavaScript for the frontend.
>
> Follow these requirements exactly:
>
> - Produce a folder hierarchy with one file per class or module shown in the diagram, organised sensibly for the stack.
> - Every file must be empty except for TODO comments describing what belongs in it.
> - Do not write any code. No imports, no struct or type declarations, no function signatures, no configuration values, no dependency lists. A TODO comment is the only permitted content in any file.
> - This applies to every file, including `Cargo.toml`, `package.json` and any config file — TODO comments only.
> - Return the result as a downloadable `.zip`.

---

2. Run 1 response — diagram supplied as PNG

> Your PolyNTU skeleton is ready in polyntu-skeleton.zip. I checked every file, and each line is a TODO comment with no code.
>
> **Layout**

Folder	What it holds	Diagram classes
`backend/src/boundary/`	Interfaces to the outside world	AdminApi, ResolverGateway
`backend/src/control/`	One file per use-case control	All 7 controls
`backend/src/entity/`	One file per domain entity	All 10 entities
`frontend/src/boundaries/`	One React page per UI boundary	AccountPanel, CreateMarketPage, MarketPage, MarketBrowsePage, PortfolioPage
Root and `docs/`	README, plus docs/developer/class-diagram.md	The matrix the diagram footer points to

The minimum glue the stack needs is also included, as TODO-only files: main.rs, the three mod.rs files, Cargo.toml, package.json, vite.config.js, index.html, main.jsx and App.jsx.

Design choices

AdminApi and ResolverGateway are in the backend. The diagram marks AdminApi as an API rather than a screen, and ResolverGateway calls an external service. Neither belongs in React.

The TODOs follow the diagram. Each entity file lists its attributes with multiplicities and its associations. Each control file lists its methods and dependencies.

Some types are not in the diagram as classes. Value types such as Money, Quantity and the *Kind/*State enums appear only as attribute types. There is a TODO in entity/mod.rs to decide where they live.

One catch: package.json

JSON has no comment syntax. Following your rule, package.json contains only // TODO lines, so npm will refuse to parse it until someone replaces it with real JSON. Cargo.toml is fine, because TOML supports # comments.

Result: 36 files. Files were produced for 24 of the 39 classes in the diagram. Every class in the `<<container>>`, `<<strategy>>`, `<<infrastructure>>` and `<<active>>` stereotypes was absent, along with `StartupControl`, `ApiRouter` and `NtuBusAdapter`.

3. Run 2 response — same diagram supplied as PlantUML source

The skeleton is in polyntu-skeleton.zip: 52 files, and a scan confirmed every line in every file is a TODO comment.

Layout

Folder	Diagram classes
`backend/src/`	main.rs (StartupControl), store.rs, app_state.rs

Folder	Diagram classes
`backend/src/api/`	ApiRouter, AdminApi
`backend/src/controls/`	The 7 use-case controls
`backend/src/domain/`	10 entities and 3 containers (OutcomeSet, PositionBook, TradeHistory)
`backend/src/resolution/`	ResolutionAuthority and its 3 strategies
`backend/src/integrations/`	ResolverGateway, NtuBusAdapter
`backend/src/runtime/`	BackgroundWorker, EventBus, EventRuntime
`frontend/src/pages/`	MarketBrowsePage, CreateMarketPage, MarketPage, PortfolioPage
`frontend/src/components/`	AccountPanel

Each TODO lists that class's operations and attributes from the diagram, plus which other classes it collaborates with.

Three things to know before you hand this out

1. package.json and Cargo.toml will not work as they are.

JSON has no comment syntax, so a package.json with only a comment in it is invalid. npm will fail to parse it. Cargo requires a [package] section, so cargo build will reject a Cargo.toml that has only comments.

I kept both files comment-only because you asked for that. Each has a TODO noting it must be replaced with real content before anyone runs a tool.

2. A few files go beyond "one per class." Some files don't correspond to a class in the diagram, but the stack needs them:

the Rust mod.rs files, main.jsx, App.jsx and index.html, vite.config.js, api/client.js

Remove them if your marker is strict about matching the diagram one-to-one. I'd keep them and explain why in the report.

3. Shared value types have no file yet. The diagram uses types like Money, Identifier, MarketRule and Resolution but never defines them. The TODO in domain/mod.rs flags that someone needs to decide where they go. Settle this early, because every entity depends on them.

Result: 52 files. All 39 classes present. No code in any file.

---

4. Critical evaluation (3.3.4)

Q1. To what extent do you agree with its choices?

We agree with most of them. The folder structure is the decision worth discussing. The AI organised the backend into `boundary/`, `control/` and `entity/` directories. This mirrors the design

model but is not how Rust projects are normally laid out, and it is not how our own repository is organised. We think it was still the right choice. A skeleton is the starting point for a project that does not exist yet, and in that context the traceability from stereotype to folder is worth more than following language convention. Section 3.4.2 of the lab manual suggests this organisation directly. We would not apply it to our existing codebase, because restructuring working code to match a diagram costs more than it gains. The second run described under the next question organised the same classes functionally rather than by stereotype, so this was not a consistent choice by the model.

Three other decisions were sound. Our diagram marks nine classes as `<<boundary>>`. Five are React pages and four are not. The AI placed `AdminApi` and `ResolverGateway` in the backend instead of treating every boundary class as a screen, which means it read the diagram's legend rather than just the box colours. It also flagged the shared value types (`Money`, `Quantity`, `Identifier` and the state enums) as a decision still to be made, instead of inventing files for them. That exposed a real gap in our own design model, since those types appear throughout the attribute lists but are never defined as classes. Lastly, it proposed a router and a charting library in `package.json`. Neither is mentioned in our diagram, so this was an inference rather than a reading, but both are reasonable for a prediction-market interface and our implementation does include price charts.

One case forced the AI to choose between two of our instructions, and we agree with how it chose. We required every file to contain nothing but TODO comments, but JSON has no comment syntax, so `package.json` cannot satisfy that rule and remain parseable. The AI followed the instruction, leaving a file that npm will reject, and stated the conflict explicitly in its response instead of quietly writing valid JSON. We would rather it surface a contradiction in our requirements than resolve one on its own. The same situation did not arise for `Cargo.toml`, because TOML supports `#` comments.

We disagree with one choice. `StartupControl`, `ApiRouter` and `BackgroundWorker` were folded into `main.rs` without being named anywhere. Their responsibilities survive as TODO comments covering configuration, route registration and the scheduled lifecycle job. Three classes from the design model leave no trace in the skeleton. Putting that logic in `main.rs` is idiomatic Rust and matches our implementation, so we have no objection to the placement. The problem is the silent omission. A developer working from this skeleton would have no way of knowing those classes exist in the design.

Q2. Has it missed something obvious, or failed to produce anything reasonable?

It missed a great deal. The skeleton contains files for 24 of the 39 classes in our diagram, and the pattern of what survived is exact. Every entity was produced, along with seven of the eight controls and seven of the nine boundaries. Every class in the four stereotypes we added during Lab 3 was dropped. That means all three containers (`OutcomeSet`, `PositionBook`, `TradeHistory`), the `ResolutionAuthority` interface and its three implementations, the four infrastructure classes (`Store`, `AppState`, `EventBus`, `EventRuntime`) and the active class `BackgroundWorker`. `StartupControl`, `ApiRouter` and `NtuBusAdapter` were also absent. In effect the output covered the analysis model we had at the end of Lab 2 and none of the design work that distinguishes Lab 3 from it.

Two consequences follow. There is no persistence layer anywhere in the skeleton, which is a serious gap for an application whose core is an exact-integer ledger. The Strategy pattern that our design documents as one of three applied patterns has no representation at all, so the way resolution authority varies between administrator, creator signature and external resolver is

invisible. More importantly, the AI did not mention any of this. Its summary described the layout it had produced and never stated that roughly a third of the diagram was absent. We consider silent incompleteness a worse failing than any single omission, because a reader has no prompt to check.

What it did produce was reasonable. The TODO comments describe each class's responsibility accurately, the collaborations it lists match the diagram, and nothing in the output is incoherent or invented. The failure is one of completeness rather than quality.

We then ran a controlled follow-up to find out why. The first run was given the diagram as a PNG, which is the format a person would naturally attach. We repeated the identical prompt in a fresh chat with the PlantUML source file instead, changing nothing else. All 39 classes appeared, across 52 files, and the new files were accurate: `outcome_set.rs` carries the two-to-eight bound on outcomes, and `resolution_authority.rs` records that `MarketLifecycleControl` selects the implementation from the series. The second run also caught a problem the first had missed, noting that `Cargo.toml` requires a `[package]` section and will not build as a comment-only file. The conclusion is that the AI did not fail to understand our design model. It could not read a 2723 by 3766 pixel image containing 39 class boxes. The practical lesson is to give these tools the diagram source rather than a rendered picture of it.

Two omissions survived the second run. Neither skeleton contains a migrations directory or a test directory, even though the project needs both. This points to a limit of the exercise rather than of the tool. A class diagram does not describe schema migrations, test suites or shared value types, so a skeleton generated from one cannot contain them. Some of what is missing was never in the input to begin with.

Q3. Did it add anything to the files beyond what was requested, such as implementation code?

It added no implementation code in either run. We checked every file in both skeletons and found no line that was not a comment. There are no imports, no struct or type declarations, no function signatures and no filled-in configuration values. The instruction held even where following it produced a broken file, which is the case for `package.json` in both runs and `Cargo.toml` in the second. We also checked the content of the comments rather than only their form. The operations named in each TODO are the operations shown on the corresponding class, and visibility was carried across: the diagram marks `Store::initialize` as private and the skeleton records it as private. No method names were invented. The only content the AI supplied of its own is the short description of what each operation does, which is what we asked the TODO comments to contain.

It did add files that do not correspond to any class in the diagram. These fall into two groups. The first is the scaffolding the stack cannot run without: the Rust `mod.rs` module declarations, `Cargo.toml`, `package.json`, `index.html`, `main.jsx`, `App.jsx` and `vite.config.js`. We accept these. A folder of Rust source files without module declarations does not compile, and a React application without an entry point does not start, so a skeleton that omitted them would not be usable as a starting point. The second run also added `api/client.js` for the browser side of the HTTP contract, which is reasonable even though no class on the diagram names it. Both runs declared these additions rather than hiding them, and the second run offered to remove them if a strict one-file-per-class reading was wanted.

Two additions in the first run were not grounded in the input. Its `package.json` proposes a router and a charting library, neither of which appears anywhere in our diagram. Both turn out to suit a prediction-market interface and our implementation does use charts, but the suggestion came from general knowledge of React projects rather than from anything it was given. The same run also created `docs/developer/class-diagram.md` with TODOs for a responsibility matrix and use-case mappings. That file is named in the caption at the foot of our diagram, so the AI read the caption and produced the document it refers to. We find this defensible but outside the brief, since the request was for a project skeleton and not for documentation.